MODULE 21 · DOMAIN 8 - TOOLS AND MCPS

Building MCP Servers

Hosts, clients and servers; tools, resources and prompts; stdio and Streamable HTTP; authoring, testing and authorisation; and connecting servers to Claude Code, the Agent SDK and the Messages API.

DOMAIN 8 of 8 EXAM WEIGHT 10.6% MODULE 21 of 36

admin@sthithapragna.com

MCP is a standard plug socket. **Build the integration once, and every compliant host can use it.**

Hosts, clients and servers

01
Host
The AI application: Claude Code, Desktop, your agent.

02
Client
A connector inside the host, one per server.

03
Server
Exposes your system's tools, resources and prompts.

04
Messages
JSON-RPC 2.0 over a transport.

05
Your system
The server calls it, with its own permissions.

ON THE EXAM

The host decides what the model sees and what needs approval. The server decides what it exposes.

What a server can offer

PRIMITIVE	WHAT IT IS	CONTROLLED BY
Tools	Functions for the AI model to execute	The model, with approval
Resources	Context and data, for the user or the model	The application
Prompts	Templated messages and workflows for users	The user
Elicitation (client)	The server asks the user for more information	The user

CURRENCY CHECK

In MCP 2026-07-28, roots, sampling and logging are deprecated. Tools, resources and prompts remain the core.

Transports: stdio or Streamable HTTP?

STDIO

A local process

- Host starts the server as a subprocess
- Messages over standard input and output
- One user, one machine
- Credentials from the environment

STREAMABLE HTTP

A remote service

- One endpoint, many clients
- Deployed like any web service
- Shared across teams and apps
- OAuth 2.1 authorisation

ON THE EXAM

The spec defines stdio and Streamable HTTP. HTTP+SSE is deprecated; WebSocket is only a custom transport some clients, such as Claude Code, accept.

A small server with the Python SDK

```python
from mcp.server import MCPServer

mcp = MCPServer("copperlet-property")

@mcp.tool()
async def property_get(property_id: str) -> str:
    """Look up one property by ID: address, landlord, active tenancy and
    certificate dates. Use for questions about a specific property."""
    record = await property_api.get(property_id)            # your REST service
    return json.dumps(record.public_fields())

if __name__ == "__main__":
    mcp.run(transport="stdio")                               # never print() here
```

5-8 Type hints and the docstring become the tool definition.

13 stdio: anything printed to stdout corrupts the protocol.

CURRENCY CHECK

"Never write to stdout" from a stdio server. Log to stderr or a file.

Testing and deploying

- **MCP Inspector:** npx @modelcontextprotocol/inspector. List and call tools, read resources, see raw messages.
- **Unit-test the handlers** like any other code, without a model.
- **Evaluate with a model:** do agents pick the right tools from your descriptions?
- **Remote servers** deploy like web services: health checks, logs, rate limits, versions.
- **Version your tools:** renaming one breaks every host that relies on it.

> THE TAKEAWAY
>
> Test the server with the Inspector first; test the tool descriptions with real agent tasks second.

Authorisation for remote servers

- **HTTP servers:** OAuth 2.1, with protected resource metadata and resource indicators.
- **Token passthrough is explicitly forbidden:** a server MUST NOT accept tokens meant for another service.
- **The server gets its own token** for the downstream API, scoped to what it needs.
- **stdio servers** take credentials from the environment instead.
- **Least privilege:** read-only tools by default; writes behind approval.

> ON THE EXAM
>
> Each hop proves who it is to the next. Passing the user's token straight through breaks that chain.

Connecting a server to Claude

WHERE	HOW
Claude Code	claude mcp add; scopes local, project (.mcp.json, approval prompt), user
Agent SDK	mcpServers option; or in-process SDK tools with create_sdk_mcp_server
Messages API	MCP connector (beta): remote URL servers, tool calls only
Claude apps	Connectors added by users or administrators

THE TIEBREAKER

The MCP connector cannot reach a local stdio server. Deploy it remotely, or run your own client.

What changed in MCP 2026-07-28

- **Stateless:** no initialize handshake and no Mcp-Session-Id; a server/discover call instead.
- **Multi round-trip requests** replace server-initiated requests.
- **Roots, sampling and logging deprecated;** tasks moved to an extension.
- **Client ID Metadata Documents** preferred over dynamic client registration.
- **SDKs negotiate,** so updated servers can still answer 2025-era clients.

> THE TAKEAWAY
>
> Stateless servers scale like ordinary web services: any instance can answer any request.

Copperlet's property-data MCP server

WHAT IT EXPOSES

Read first

- About 40 REST endpoints as ~12 consolidated tools
- Tenancy agreements as resources
- A "repair summary" prompt for agents
- Write tools behind approval

HOW IT RUNS

Remote, authorised

- Streamable HTTP, deployed per environment
- OAuth 2.1; its own scoped token downstream
- Used by the assistant, the agency-ops agent, Claude Code
- Tool names versioned and reviewed

THE TAKEAWAY

One server, three hosts. The integration is built and secured once.

> A Python stdio MCP server works when tested with direct function calls. When Claude Desktop connects to it, the connection fails with protocol parse errors. The code prints debugging messages as it starts.
>
> Nothing else seems wrong.

What is the most likely cause?

A The server must use Streamable HTTP to work with Claude Desktop

B The tool descriptions are too long for the client to parse

C The print statements write to stdout and corrupt the protocol messages

D Claude Desktop needs a WebSocket transport

◆ PAUSE AND DECIDE

What is the most likely cause?

A The server must use Streamable HTTP to work with Claude Desktop

Desktop runs local stdio servers.

B The tool descriptions are too long for the client to parse

Length is not a parse error.

C The print statements write to stdout and corrupt the protocol messages

The documented stdio pitfall.

D Claude Desktop needs a WebSocket transport

Not a standard transport.

THE SIGNAL

stdio server, parse errors, print statements: stdout is the protocol channel.

> Three teams each built their own Claude integration with the company's ticketing system: one in a chatbot, one in an internal agent, one for engineers in Claude Code. The three integrations behave differently and duplicate effort.
>
> More teams want the same access.

What is the best approach?

A Build one ticketing MCP server, and connect each host to it

B Let each team keep its own integration, with a shared style guide

C Give every team direct database credentials for the ticketing system

D Paste the ticketing API documentation into every system prompt

◆ PAUSE AND DECIDE

What is the best approach?

A Build one ticketing MCP server, and connect each host to it

Built, secured and fixed once.

B Let each team keep its own integration, with a shared style guide

Keeps the duplication.

C Give every team direct database credentials for the ticketing system

Broad access, no shared tools.

D Paste the ticketing API documentation into every system prompt

Documentation is not an integration.

THE SIGNAL

The same system, many hosts, more coming: that is what MCP is for.

> A remote MCP server for a finance API receives the user's OAuth access token from the MCP client, and forwards that same token to the finance API on each call.
>
> A security review flags the design.

What should the team change?

A Encrypt the user's token before forwarding it to the finance API

B Move the server to stdio, so tokens never cross the network

C Log every forwarded token, so misuse can be traced later

D Accept only its own tokens, and use its own scoped credential downstream

◆ PAUSE AND DECIDE

What should the team change?

A. Encrypt the user's token before forwarding it to the finance API

Still passthrough, just encrypted.

B. Move the server to stdio, so tokens never cross the network

Breaks the shared remote service.

C. Log every forwarded token, so misuse can be traced later

Puts secrets in logs, and still forwards them.

D. Accept only its own tokens, and use its own scoped credential downstream

No passthrough; least privilege.

THE SIGNAL

"Forwards that same token" is token passthrough, which the MCP specification explicitly forbids.

> A backend service calls the Messages API and wants Claude to use an MCP server. The server currently runs only as a local stdio process on a developer's laptop.
>
> The team plans to use the MCP connector.

What must the team do?

A Point the connector at the stdio command on the laptop

B Deploy the server remotely over Streamable HTTP, or run an MCP client in their own code

C Paste the server's code into the system prompt

D Switch the server to a WebSocket transport, which the connector prefers

◆ PAUSE AND DECIDE

What must the team do?

A Point the connector at the stdio command on the laptop

Local stdio servers cannot be connected.

B Deploy the server remotely over Streamable HTTP, or run an MCP client in their own code

The two routes that work.

C Paste the server's code into the system prompt

Code in a prompt runs nothing.

D Switch the server to a WebSocket transport, which the connector prefers

Not what the connector uses.

CURRENCY CHECK

The MCP connector reaches remote URL servers only. Local stdio needs your own client.

> A company is publishing an MCP server for its booking system to many internal teams. Some teams' agents will only read availability; a few must create bookings.
>
> The team wants the server to be safe and dependable.

Which two practices fit best? Choose two.

A Expose every internal endpoint as its own tool, so nothing is missing

B Keep read tools separate from write tools, with writes requiring approval and narrower scopes

C Share one administrator credential across all clients for simplicity

D Rename tools freely as the backend changes

E Test with the Inspector and agent evals, and version tool names and schemas

◆ PAUSE AND DECIDE

Which two practices fit best? Choose two.

A Expose every internal endpoint as its own tool, so nothing is missing

Too many overlapping tools.

B Keep read tools separate from write tools, with writes requiring approval and narrower scopes

Least privilege, by tool.

C Share one administrator credential across all clients for simplicity

Maximum blast radius.

D Rename tools freely as the backend changes

Breaks every host that relies on them.

E Test with the Inspector and agent evals, and version tool names and schemas

Dependable for many consumers.

THE SIGNAL

Safe: separate reads from writes, with approval and scopes. Dependable: tested and versioned.

> A team runs its remote MCP server on several instances behind a load balancer. Under an older protocol version, requests failed when they reached an instance that had not seen the session's initialize handshake.
>
> The team is updating to the current specification.

Why does the current specification help?

A. It is stateless, so any instance can answer any request

B. It requires a single server instance per client

C. It moves all servers to stdio, avoiding load balancers

D. It pins each client to one instance using WebSockets

◆ PAUSE AND DECIDE

Why does the current specification help?

A. It is stateless, so any instance can answer any request
The 2026-07-28 change.

B. It requires a single server instance per client
The opposite of the change.

C. It moves all servers to stdio, avoiding load balancers
Streamable HTTP remains standard.

D. It pins each client to one instance using WebSockets
Not in the specification.

CURRENCY CHECK

Stateless protocol, stateless instances: scale out behind a load balancer like any web service.

Module 21 in five lines

- **Host, client, server;** build an integration once, use it from many hosts.
- **Tools, resources, prompts;** roots, sampling and logging now deprecated.
- **stdio locally, Streamable HTTP remotely;** never write to stdout in stdio.
- **OAuth 2.1 for remote servers;** token passthrough is forbidden.
- **MCP 2026-07-28 is stateless;** the API connector reaches remote servers only.

COMING UP

Next: built-in tool, custom tool, Skill or MCP server - choosing the right extension.